

Data-Driven Design & Analysis of Structures & Materials (3dasm)

Instructor: Miguel A. Bessa

Brown University

Homework 5

Deliver a **ZIP folder with two files (PDF and Jupyter notebook)** containing the answers to the questions of this assignment. **UPLOAD to CANVAS in the Assignments section (Homework 5) by the due date.**

Due date: until 11:59pm of day announced in [course home page](#).

This Homework aims to help you reason about regularization, the bias–variance trade-off, Bayesian linear regression and heteroscedastic noise (Lectures 11 to 13), using an AI coding agent to write and run the code. This is a homework containing questions to be solved with **pen and paper** as well as with **code**: the notebook you submit contains your prompts, the code the agents wrote, their results and your answers.

How to work. Start from the notebook `Assignments/3dasm_Homework5.ipynb`: copy it to `Assignments/your_Assignments/HW5/your_Homework5.ipynb` and work there. As in the lectures, an AI coding agent writes and runs the code, and the notebook is the source of truth: you write the agent's instructions file (`CLAUDE.md`) in a notebook cell, each prompt you give the agent is a notebook cell, and the cells that we provide load each agent's `results.json` from its task folder.

- **Model.** Use Sonnet or Haiku (`set_model("sonnet")` or `set_model("haiku")`); the course's helper does not accept other models.
- **Grading.** The TA runs your notebook **from scratch** (every agent runs again), with the model you declare in it. The regenerated numbers must be the same as the ones you submitted: if they are not, your instructions left something for the agent to guess. Your numbers are also compared with the solution: the homework fixes the dataset, the split and the models, so a precise notebook reproduces them exactly. Your predictions, checks and interpretations are graded as usual; base them on the numbers and figures your notebook stores, not on the agent's summary.
- **Prompts** are written in words. Short expressions that pin the data, the randomness or the method are fine (e.g. `random_state=123`); code that implements the task is not.
- **Your prompts and your instructions file are graded.** In the exercises with the agent, a good prompt defines the problem **uniquely**: the data, the randomness (seeds, and the order of the draws), the model and the method, and what to store and plot. A prompt that leaves choices open makes the outcome depend on what the agent happens to guess, which can change from one run to the next. The TA re-runs your notebook with fresh agents: a number that changes counts as an error, even if your answer was right. The TA also reads your prompts: an ambiguity that happened to reproduce counts as a minor error.
- **Write your prompts in your own words.** A prompt that only points to a question (e.g. "solve exercise 4.2") or that pastes it is not a prompt: that item counts as not answered. You may copy the specifications (numbers, seeds, names of functions).
- Each exercise that says (**pen and paper**) should not be resolved with an AI agent (remember that you won't have access to AI in the midterm, so it's better for you not to use it for these questions)
- Each exercise that says (**with the agent**) is appropriate to resolve with an agent as we introduced in the lectures.
 - Note: **writing your own code instead of using the agent is allowed** for the exercises signalled as (**with the agent**). Put your code in the same notebook cell (not in a separate file).
- **Before you submit**, delete the task folders and run the notebook from the start, so that its outputs come from one clean run (this is also what the TA will do when grading).

- **Submit one ZIP file** with your notebook (.ipynb), which the TA re-runs, and a PDF of that same notebook after your clean run. If you write the pen and paper answers by hand, add a scan of them as a third file. Do not include the task folders (they are not needed: the notebook is the source of truth).

Exercise 1 (pen and paper)

Consider Least Squares, Ridge and Lasso regression as derived in class, with M basis functions that are **orthonormal on the training points**, $\Phi^T \Phi = \mathbf{I}_M$, and no separate bias term. Each method gives a **point estimate** of the weights: $\hat{\mathbf{w}}_{\text{LS}}$ (the MLE), $\hat{\mathbf{w}}_{\text{ridge}}$ and $\hat{\mathbf{w}}_{\text{lasso}}$ (MAP estimates). In this exercise you compare them analytically.

1. Why Ridge shrinks and Lasso selects.

- 1.1. Show that $\text{RSS}(\mathbf{w}) = \text{RSS}(\hat{\mathbf{w}}_{\text{LS}}) + \|\mathbf{w} - \hat{\mathbf{w}}_{\text{LS}}\|_2^2$, where here $\text{RSS}(\mathbf{w}) = \|\mathbf{y} - \Phi \mathbf{w}\|_2^2$. (Hint: write $\mathbf{y} - \Phi \mathbf{w} = (\mathbf{y} - \Phi \hat{\mathbf{w}}_{\text{LS}}) + \Phi(\hat{\mathbf{w}}_{\text{LS}} - \mathbf{w})$, and use the condition that defines the Least Squares estimate in Lecture 9, $\Phi^T \Phi \hat{\mathbf{w}}_{\text{LS}} = \Phi^T \mathbf{y}$.) This result says that the RSS of any $\mathbf{w}$ is the smallest possible RSS plus the squared distance between $\mathbf{w}$ and $\hat{\mathbf{w}}_{\text{LS}}$: Ridge and Lasso trade this distance against their penalty, which lets you find and compare their point estimates in closed form.
- 1.2. Using 1.1., show that the point estimate of Ridge (minimizing $\text{RSS}(\mathbf{w}) + \alpha \|\mathbf{w}\|_2^2$) is $\hat{\mathbf{w}}_{\text{ridge}} = \hat{\mathbf{w}}_{\text{LS}} / (1 + \alpha)$. (Hint: Lecture 9 used $\frac{\partial \mathbf{u}^T \mathbf{A} \mathbf{u}}{\partial \mathbf{u}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{u}$; here $\mathbf{A} = \mathbf{I}$.) Can a weight become exactly zero?
- 1.3. Using 1.1., show that the point estimate of Lasso (minimizing $\text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$) is, for each weight, $\hat{w}_{\text{lasso},m} = \text{sign}(\hat{w}_{\text{LS},m}) \max(|\hat{w}_{\text{LS},m}| - \lambda/2, 0)$, called **soft-thresholding**. (Hint: write the objective with indices, $\|\mathbf{w} - \hat{\mathbf{w}}_{\text{LS}}\|_2^2 = \sum_m (w_m - \hat{w}_{\text{LS},m})^2$ and $\|\mathbf{w}\|_1 = \sum_m |w_m|$: it becomes a sum of M problems with one weight each. For one weight, consider $w_m > 0$, $w_m < 0$ and $w_m = 0$.) Sketch the point estimate $\hat{w}_m$ as a function of $\hat{w}_{\text{LS},m}$ for Ridge and for Lasso.
- 1.4. As λ grows, which weights does Lasso set to zero first? What happens to the weights that remain non-zero? Relate both to the shape of the Laplace prior compared with the Gaussian prior.
- 1.5. In Ridge regression with a complete polynomial basis of degree p , which quantities are parameters and which are hyperparameters? Why should the hyperparameters be chosen with cross-validation on the training set, and not with the test set?

Exercise 2 (pen and paper)

Consider Linear Least Squares with M basis functions and N training points at fixed inputs $\mathbf{x}_1, \dots, \mathbf{x}_N$, with the matrix of basis functions Φ ($N \times M$, with $\Phi^T \Phi$ invertible). Assume that the model can represent the true function, $f(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w}$, so that $\mathbf{y} = \Phi \mathbf{w} + \boldsymbol{\epsilon}$, with independent noise $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$.

2. The variance of the predictions of Linear Least Squares.

- 2.1. Show that the point estimate $\hat{\mathbf{w}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$ is unbiased, i.e. that $\mathbb{E}[\hat{\mathbf{w}}] = \mathbf{w}$, and that its covariance matrix is $\text{Cov}[\hat{\mathbf{w}}] = \sigma^2 (\Phi^T \Phi)^{-1}$. (Hint: $\text{Cov}[\hat{\mathbf{w}}] = \mathbb{E}[(\hat{\mathbf{w}} - \mathbf{w})(\hat{\mathbf{w}} - \mathbf{w})^T]$ and $\mathbb{E}[\boldsymbol{\epsilon} \boldsymbol{\epsilon}^T] = \sigma^2 \mathbf{I}_N$.)
- 2.2. Show that the variance of the prediction $\hat{y}(\mathbf{x}) = \phi(\mathbf{x})^T \hat{\mathbf{w}}$ at an input $\mathbf{x}$ is $\sigma^2 \phi(\mathbf{x})^T (\Phi^T \Phi)^{-1} \phi(\mathbf{x})$. What is the bias of this prediction, $\text{Bias}[\hat{y}(\mathbf{x})] = \mathbb{E}[\hat{y}(\mathbf{x})] - f(\mathbf{x})$?
- 2.3. The variance of 2.2., averaged over the N training inputs, is $\sigma^2 M/N$ (if you are curious: each term is a number, equal to its own trace, and $\text{tr}(\mathbf{A}\mathbf{B}) = \text{tr}(\mathbf{B}\mathbf{A})$), and the expected training MSE is $\sigma^2(N - M)/N$ (Lectures 9 and 11). Now consider new observations at the same inputs, $y_n^* = f(\mathbf{x}_n) + \epsilon_n^*$, with new noise ϵ_n^* independent of the training data. Show that the expected MSE of the predictions on these new observations is $\sigma^2(1 + M/N)$. (Hint: write $y_n^* - \hat{y}(\mathbf{x}_n) = \epsilon_n^* + (f(\mathbf{x}_n) - \hat{y}(\mathbf{x}_n))$; this is the decomposition of the error on unseen data of Lecture 11.) Compare the expected training MSE with the expected MSE on new observations: what happens to each when M grows with N fixed, and when N grows with M fixed? Which terms of the decomposition (noise, bias², variance) appear here, which one is missing, and why?

Exercise 3 (pen and paper)

Consider Bayesian linear regression with M basis functions, $f(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$, and the prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \hat{\sigma}_w^2 \mathbf{I}_M)$. You will need the following property, which follows from the rules of Lecture 3 (the expected value is linear; covariances transform with the matrix): if $\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\mathbf{A}$ is a fixed matrix, then $\mathbf{Aw} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^T)$.

3. A prior over the weights is a prior over functions.

- 3.1. Show that the function values at N inputs, $\mathbf{f} = \boldsymbol{\Phi}\mathbf{w}$, follow $\mathcal{N}(\mathbf{f} | \mathbf{0}, \hat{\sigma}_w^2 \boldsymbol{\Phi}\boldsymbol{\Phi}^T)$. This result shows that, for two inputs $\mathbf{x}$ and $\mathbf{x}'$, the function values $f(\mathbf{x})$ and $f(\mathbf{x}')$ are jointly Gaussian, with zero mean and $\text{Cov}[f(\mathbf{x}), f(\mathbf{x}')] = \hat{\sigma}_w^2 \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$.
- 3.2. The rank of the $N \times N$ covariance matrix $\hat{\sigma}_w^2 \boldsymbol{\Phi}\boldsymbol{\Phi}^T$ is $\text{rank}(\boldsymbol{\Phi}) \leq \min(N, M)$. What does this imply, when $N > M$, about the functions that this prior can generate?
- 3.3. Write the variance of the PPD at a new input $\mathbf{x}^*$ (Lecture 13), and identify its aleatoric and its epistemic part. What happens to each part when the number of training points grows? (Hint: $\boldsymbol{\Phi}^T \boldsymbol{\Phi} = \sum_n \boldsymbol{\phi}(\mathbf{x}_n) \boldsymbol{\phi}(\mathbf{x}_n)^T$ has one term per training point; for training points spread over the same domain, it grows like $N\mathbf{C}$, where $\mathbf{C}$ is a fixed matrix, the average of $\boldsymbol{\phi}(\mathbf{x}) \boldsymbol{\phi}(\mathbf{x})^T$ over the inputs.) And what happens to each part when $\hat{\sigma}_w \rightarrow 0$?

Exercise 4 (with the agent)

In this exercise you measure the decomposition of the error on unseen data seen in class, by drawing many training sets. Consider $f(x) = x \sin(x)$ in $[0, 10]$ and noise with standard deviation $\sigma = 1$. A **training set** of size N has the inputs $x = \text{np.linspace}(0, 10, N)$ (always the same inputs) and the outputs $y = f(x) + \epsilon$, with a new noise draw $\epsilon \sim \mathcal{N}(0, 1)$ for every training set. For each N , draw 1000 training sets one after the other from a new NumPy default random number generator with seed 0 (the noise of training set r is the r -th draw of N values; training set 1 is the **first training set**). Use polynomial models with the inputs rescaled to $[-1, 1]$, and evaluate every model on 201 equally spaced points of $[0, 10]$.

4. Bias and variance, measured.

- 4.1. **Before running anything**, sketch how you expect the bias², the variance and their sum with the noise to change with the polynomial degree, for $N = 20$ and for $N = 80$.
- 4.2. For $N = 20$ and $N = 80$, and for degrees 1, 3, 5, 7, 9, 12 and 15 (Linear Least Squares), estimate the bias² and the variance of the predictions (each averaged over the 201 evaluation points), and the expected error on new observations, noise + bias² + variance. For degree 7 and $N = 20$, check this sum against the MSE of the predictions on new noisy observations at the evaluation points (one draw of 201 noise values per training set, from a new generator with seed 1). Plot the three curves against the degree for both N .
- 4.3. Check the average variance $\sigma^2 M/N$ of 2.3.: for degrees 3 and 7 with $N = 20$, and 7 and 12 with $N = 80$, compare the variance averaged over the **training inputs** with $\sigma^2 M/N$.
- 4.4. For degree 12 and $N = 20$, repeat 4.2. with Ridge regression for $\alpha = 10^{-6}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1$ and 10, and plot the curves against α .
- 4.5. Now use only the **first** training set of each size, as you would in practice: choose the degree with 5-fold cross-validation (points shuffled, seed 123). Which degree does it choose for $N = 20$ and for $N = 80$, compared with the minimum of 4.2.?
- 4.6. For degree 12 and $N = 20$, repeat the variance estimate with inputs taken from a single uniform draw of 20 values in $[0, 10]$ (a NumPy default random number generator with seed 123, kept fixed for all training sets) instead of equally spaced; the noise is drawn as before. Plot the variance against x for both choices of inputs. Where is it large, and why?
- 4.7. **What can you conclude?** Compare with your predictions of 4.1..

Exercise 5 (with the agent)

In this exercise the noise changes with the input (heteroscedastic noise, Lecture 13), and you know the truth, so you can check every step. Consider $f(x) = 2\sin(x) + 0.5x$ in $[0, 10]$ and noise with standard deviation $\sigma(x) = 0.2e^{0.25x}$. A **training set** with seed s has $N = 60$ inputs drawn uniformly in $[0, 10]$ and outputs $y = f(x) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2(x))$, both from a NumPy default random number generator with seed s (first the 60 inputs, then the 60 noise values). Unless said otherwise, use the training set with seed 0. The **test set** has 5000 points drawn in the same way with seed 1. The model of the mean is Bayesian linear regression with a polynomial of degree 7 in $\tilde{x} = x/5 - 1$ (8 basis functions, including the constant) and the prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, 10^2\mathbf{I})$ on all the weights. The **coverage** of a model is the fraction of test points with $|y - \text{PPD mean}| \leq 1.96$ PPD standard deviation, computed separately in each third of the domain, $[0, 10/3]$, $[10/3, 20/3]$ and $[20/3, 10]$.

5. Heteroscedastic noise, with the truth known.

- 5.1. **Before running anything**, predict the coverage in each third for a model that assumes one noise level σ for all points.
- 5.2. Fit the model with one noise level (homoscedastic Bayesian linear regression), estimating σ by alternating: start with $\sigma = 1$ and repeat 50 times: compute the posterior of $\mathbf{w}$ with the current σ , then update $\hat{\sigma}^2 = \sum_n (y_n - \hat{\mu}(x_n))^2 / (N - M)$, where $\hat{\mu}$ is the PPD mean. Report $\hat{\sigma}$ and the coverage in each third, and plot the training points, f , the PPD mean and two bands of ± 2 standard deviations (aleatoric only, and aleatoric + epistemic).
- 5.3. Now model the noise with $\log \sigma^2(x) = v_0 + v_1 \tilde{x}$, and solve the model with the staggered scheme of Lecture 13: start from the result of 5.2. ($v_0 = \log \hat{\sigma}^2$, $v_1 = 0$) and repeat 10 times: (i) the **noise step**: minimize, with 100 steps of Newton's method, $\frac{1}{2} \sum_n \left[\mathbf{v}^T \boldsymbol{\psi}(x_n) + \tilde{r}_n^2 e^{-\mathbf{v}^T \boldsymbol{\psi}(x_n)} \right]$, where $\boldsymbol{\psi}(x) = [1, \tilde{x}]$ and $\tilde{r}_n^2 = (y_n - \hat{\mu}(x_n))^2 + \boldsymbol{\phi}(x_n)^T \hat{\boldsymbol{\Sigma}}_w \boldsymbol{\phi}(x_n)$ is the squared residual averaged over the posterior of the weights; (ii) the **mean step**: the posterior of $\mathbf{w}$ with the noise variances $\sigma^2(x_n) = e^{\mathbf{v}^T \boldsymbol{\psi}(x_n)}$. Report the estimated $\hat{\sigma}(x)$ at $x = 0, 5$ and 10 after each iteration, the largest change of the PPD mean (on 201 equally spaced points of $[0, 10]$) from one iteration to the next, and the final coverage in each third. Plot $\hat{\sigma}(x)$ after each iteration together with the true $\sigma(x)$, and the final PPD as in 5.2.
- 5.4. Repeat 5.2. and 5.3. for the 200 training sets with seeds 0 to 199 (the same test set), and report the coverage in each third averaged over the 200 training sets for: one noise level (5.2.); the staggered scheme with the plain squared residual $(y_n - \hat{\mu}(x_n))^2$ in the noise step; and the staggered scheme with $\tilde{r}_n^2$ (5.3.). Why is the coverage lower with the plain squared residual? (Hint: Lecture 11, the training MSE.)
- 5.5. **What can you conclude?** Compare with your predictions of 5.1.. Which part of the uncertainty does each step of the staggered scheme estimate? How much does the mean change from one iteration to the next, and how quickly does the scheme converge?

Exercise 6 (with the agent)

In practice nobody tells you the truth. The file `Assignments/data/HW5_measurements.csv` has 1000 measurements (x, y) , with $x \in [0, 10]$, of a quantity whose mean and noise we do not reveal. Split them into training and test sets with `train_test_split(test_size=0.25, random_state=123)`, and use the test set only to evaluate. Here the coverage is computed in each of five bins of the test set, $[0, 2]$, $[2, 4]$, $[4, 6]$, $[6, 8]$ and $[8, 10]$.

6. When the truth is unknown.

- 6.1. Fit, on the training set, the two models of Exercise 5 with exactly the same choices (a polynomial of degree 7 in $\tilde{x} = x/5 - 1$ for the mean, the prior $\mathcal{N}(\mathbf{0}, 10^2\mathbf{I})$, $\boldsymbol{\psi}(x) = [1, \tilde{x}]$ for the log-variance, and 10 iterations of the staggered scheme with $\tilde{r}_n^2$). Report, for both, the coverage in each bin and the test MSE of the PPD mean.
- 6.2. **Diagnose** the models of 6.1. using only the training set: plot the residuals of the PPD mean of the staggered model, and the squared residuals $\tilde{r}_n^2$ (e.g. on a logarithmic scale), against x , together with the estimated $\hat{\sigma}^2(x)$. What is wrong with the model of the mean, and what is wrong with the model of the noise? Justify with your figures.

- 6.3. **Improve** the model: choose the polynomial degree of the mean with 5-fold cross-validation of Linear Least Squares on the training set (`KFold`, points shuffled, `random_state=123`), and choose a basis ψ for the log-variance that you justify with your diagnosis. Report the degree, the coverage in each bin and the test MSE, and plot the final PPD. Is the result perfect? If not, what limits it?
- 6.4. **What can you conclude?** How would you know that a model of the noise is wrong if nobody gave you the truth?

Exercise 7 (optional, with the agent)

This exercise is optional: do it if you finished the others. It helps us gauge the workload of the homework.

Consider 30 points on the line $y = 1 + 0.5x$: $x = \text{np.linspace}(0, 10, 30)$ and $y = 1 + 0.5x + \epsilon$, with $\epsilon = \text{np.random.default_rng}(0).normal(0, 0.5, 30)$. Then add 10 to the outputs of the points with indices 25, 26 and 27 (three outliers).

7. Outliers: a question of likelihood, not of prior.
 - 7.1. **Before running anything**, predict which of the models below will recover the line $y = 1 + 0.5x$ best, and why.
 - 7.2. Fit a straight line with scikit-learn's `LinearRegression`, `Ridge(alpha=10)`, `Lasso(alpha=0.5)`, `HuberRegressor()` and `QuantileRegressor(quantile=0.5, alpha=0, solver="highs")`. Report the slope, the intercept and the MSE between each fitted line and the true line on 201 equally spaced points of $[0, 10]$, and plot the data with the five lines.
 - 7.3. **What can you conclude?** Compare with your predictions of 7.1.. Why do Ridge and Lasso not help? Which observation distribution (likelihood) corresponds to `QuantileRegressor` with `quantile=0.5`, and how does `HuberRegressor` treat the points far from the line?